

AI 时代，嵌入式工程师的职业规划

——关于“是否深耕·如何深耕·学什么·怎么学”的问答整理·2026年9月14日

【提问】

我是一名程序员，同时也是一名嵌入式软件工程师。现在 AI 都可以直接接入并调试设备了，之前学习积累的编程语言、RTOS 原理、Linux 内核知识等因 AI 编程变得简单易得，职业生涯不知道该如何规划了。嵌入式行业是否值得继续深耕，该如何深耕，学什么，该如何学？

【回答】

先说结论：嵌入式值得继续深耕，而且它比纯软件开发更“抗 AI”。但你必须主动向上迁移——从“写驱动、调 RTOS 的人”变成“定义系统、打通软硬件、对可靠性负责的人”。

一、为什么嵌入式反而更抗 AI

AI 写代码的能力主要压缩的是“信息世界”的工作，而嵌入式的一半工作在“物理世界”：

硬件 bring-up 和联调没有捷径。示波器、逻辑分析仪、时序问题、信号完整性、电源抖动、EMC——这些需要手碰到板子、眼看到波形。AI

可以接入设备调试，但它依赖你先把问题空间定义清楚，且实验室里大量问题是“非标准化”的。

实时性和可靠性无法靠生成代码兜底。一个 RTOS 任务优先级配置错误，常温测试一切正常，高低温下死机。能判断“这段 AI 生成的驱动代码在 -40℃ 还会不会工作”的，是有原理功底的人。

认证和合规是人签字的。车规 ISO 26262、工控 IEC 61508、医疗 IEC 60601——文档、流程、安全分析，责任主体是人。

硬件资源约束本身构成护城河。64KB Flash 的 MCU 上跑不了大模型，裁剪、优化、汇编级调优，AI 可以辅助但拍板靠人。

但也要诚实：被商品化的是——驱动模板、简单应用逻辑、协议栈的常规使用、RTOS 移植的套路、Linux 内核常规驱动开发。如果价值只在这里，确实危险。

二、职业规划：三条主线可并行

① 深度技术线：成为“软硬件边界上的人”——系统架构、BSP 与内核定制、启动流程与安全启动、功耗与性能优化、硬件-软件协同设计。AI 生成的每一行代码，最终要在一块具体的板子上可靠工作，这个“具体”没有人比你更懂。

② 增长最快的新赛道：端侧 AI——大模型正在往设备端下沉：NPU/DSP/MCU 上的 TinyML、模型量化（INT8/INT4）、推理框架适配（TFLite Micro、ONNX Runtime、厂商 NPU SDK）。纯 AI

工程师不懂寄存器和内存布局，纯嵌入式工程师不懂模型——两个都懂的人极度稀缺。

③ 领域专家线——汽车电子、工业控制、医疗电子、机器人，每个领域都有几十年的领域知识和合规门槛，AI 训练数据覆盖不到现场的边角案例。

三、学什么（按优先级）

系统架构与方法论：实时系统设计、可靠性工程、DFMEA、可观测性（日志、trace、OTA 远程诊断体系）。十年经验的浓缩都在这里。

端侧 AI 部署：学量化与推理优化，拿一块带 NPU 的开发板（瑞芯微、地平线、ESP32-S3 起步）把一个小模型真正跑起来、压到目标时延和功耗。这是未来五年嵌入式溢价最高的技能组合。

功能安全与认证流程（汽车/工控/医疗方向）：哪怕不考认证，懂流程也会让你在设计评审中脱颖而出。

内核与硬件原理不要丢：不是为手写，而是为了审查 AI 生成的代码、判断它在极端工况下是否成立——这是你的“判断力底盘”。

四、怎么学（嵌入式学习必须带“电”）

做端到端的真实项目：需求 → 选型 → 原理图评审 → bring-up → 驱动 → 应用 → 量产问题，哪怕只是一个自制小产品。刷题式学习在 AI 时代贬值最快。

把 AI 当“坐在旁边的资深同事”：让它写第一版驱动，你追问边界情况、时序约束、错误路径；你负责在板子上验证。用 AI 的产出速度放大你的验证能力，而不是替代你的思考。

每个项目沉淀一份“能讲给别人听”的笔记：能讲清楚才说明吃透了——这也是未来面试和带团队的核心资产。

五、分阶段路线

未来 1 年：保住现有工作盘，业余时间完成一个“MCU/NPU 端侧 AI”项目；开始用 AI 结对方式工作，刻意训练审查和验证的速度。

未来 2-3 年：在公司主动承担系统架构或跨软硬件模块，攒 1-2 个拿得出手的量产项目经历；行业合适则补安全认证知识。

长期：把定位从“嵌入式软件工程师”升级为“智能设备系统工程师”——软件、硬件、AI 模型、产品，负责让它们在真实物理世界中可靠协同。这个位置，AI 在可预见的未来替代不了。

一句话总结：AI 淘汰的是“只会按规格写代码的嵌入式程序员”，而嵌入式行业里最值钱的从来就不是写代码，是让一个物理设备在真实世界里可靠地工作。这门手艺没有贬值——随着设备越来越“智能”，懂得在硅片层面驾驭 AI 的人会更稀缺。